Solución para error Git de tamaño excesivo

https://gemini.google.com/app/e9d80a43ff97b7f4

Response: En el archivo image_449282.png, la terminal Git Bash muestra los resultados de varios comandos de control de versiones y un fallo crítico al ejecutar `git push origin main`: el servidor rechaza la subida con el mensaje `remote: fatal: pack exceeds maximum allowed size (2.00 GiB)`.

De acuerdo con los registros de tu cuaderno, anteriormente experimentaste problemas de desconexión de red (`early EOF`) y fallos de empaquetado al intentar subir múltiples archivos y documentos pesados, como las plantillas y guías del Cornell Bitcoin Club. Aunque en su momento la solución sugerida fue aumentar el búfer de envío de Git, este nuevo error indica un problema distinto: el volumen total de los archivos contenidos en tu historial de commits local supera el límite máximo estricto de 2 GB establecido por los servidores de GitHub para un solo envío.

Para resolver este bloqueo, aplica los siguientes pasos en tu terminal:

- **Identificar archivos masivos:** Es probable que en algún commit local reciente hayas añadido accidentalmente carpetas enteras de gran tamaño (como bases de datos, entornos virtuales, ISOs o videos).
- **Deshacer el commit problemático:** Si el problema se generó en los últimos commits que aún no has subido, puedes retroceder tu historial local sin perder las modificaciones en tus archivos ejecutando `git reset --soft HEAD~1`. Esto te permitirá desmarcar los archivos masivamente pesados.
- **Ajustar el archivo `.gitignore`:** En la imagen se observa que el archivo "Configuración de Proyectos Xiima-Labs.pdf" (de aproximadamente 8.2 MB) está sin rastrear (*untracked*), pero debes confirmar que no existan documentos mucho más pesados ya indexados. Añade cualquier archivo que supere los 100 MB a tu `.gitignore`.
- **Subir los cambios por lotes:** Si tienes un historial legítimo con muchos archivos válidos que juntos suman más de 2 GB, deberás realizar envíos divididos. Haz un *push* de commits más antiguos primero (ej. `git push origin <hash-del-commit>:main`) y ve avanzando poco a poco hasta sincronizar la rama completa, evitando enviar los 2 GB en un solo comando.

User prompt: YA TENGO EN LA NUBE DE GITHUB parte de lo que queria sincronizar. La carpeta o contendor docs tiene subcarpetas con el protocolo de ingenieria de directorios de Linux Fundation y dentro de la carpeta 02-estado-del-arte esta la carpeta o contenedor guia-cbc y en la subcarpeta "Meeting Recordings" estan los archivos mas pesados que creeria tocaria subir uno por uno paso a paso para no tener problemas. Que me sugieres?

Response: Para ejecutar la Fase 3 de manera segura y evitar el bloqueo por el límite de 2 GB, debemos subir el contenido de la carpeta `docs` fragmentándolo y dejando la subcarpeta `Meeting Recordings` aislada. Es fundamental recordar que GitHub tiene un límite estricto de **100 MB por archivo individual** en su plan estándar; si alguna de las grabaciones supera este peso, la plataforma rechazará el envío inmediatamente, incluso si lo haces uno por uno.

Aplica el siguiente protocolo paso a paso en tu terminal Git Bash:

Paso 1: Subir las carpetas ligeras de documentación Comienza sincronizando las demás carpetas de `docs` que no contienen la multimedia masiva.

```
git add docs/01-manifiesto docs/03-tesis docs/04-workshops docs/05-registros-historicos
git commit -m "docs: fase 3.1 - manifiestos, tesis, workshops y registros"
git push origin main
```

Paso 2: Subir el contenido en texto y PDF de la guía del CBC Ahora, indexaremos selectivamente el contenido de `guia-cbc`, omitiendo de forma intencional el contenedor de grabaciones.

```
git add "docs/02-estado-del-arte/guia-cbc/Founding Documents"
git add "docs/02-estado-del-arte/guia-cbc/Lesson Plans & Meeting Slides"
git add "docs/02-estado-del-arte/guia-cbc/Resources"
git add "docs/02-estado-del-arte/guia-cbc/Team — Cornell Bitcoin Club.pdf"
git commit -m "docs: fase 3.2 - recursos, manuales y documentos de guia-cbc"
git push origin main
```

Paso 3: Estrategia para "Meeting Recordings" Dado que indicas que los archivos multimedia en esta carpeta superan las 4 GB en conjunto, alojarlos directamente en un repositorio de código no es una práctica óptima. Tienes dos caminos:

- **Opción A (La más eficiente y recomendada):** GitHub no es un disco duro para videos. La mejor práctica de ingeniería es subir estos recursos pesados a una plataforma nativa de multimedia (como YouTube Oculto, Internet Archive o una carpeta de OneDrive/Google Drive institucional). Una vez subidos, crea un archivo de texto (`README.md`) dentro de `Meeting Recordings` con los enlaces directos a los videos. Finalmente, bloquea la subida de los archivos pesados locales agregando la línea `docs/02-estado-del-arte/guia-cbc/Meeting Recordings/*.mp4` (o la extensión correspondiente) a tu archivo `.gitignore`.
- **Opción B (Subida manual archivo por archivo):** Solo es viable si **cada archivo pesa estrictamente menos de 100 MB.** Si cumples esta condición, súbelos individualmente así:

  ```
  git add "docs/02-estado-del-arte/guia-cbc/Meeting Recordings/nombre_del_video_1.mp4"
  git commit -m "docs: agregando grabacion 1"
  git push origin main
  ```

  Repite este bloque para cada archivo hasta completar el directorio. (Nota: Aunque existe la herramienta Git LFS para archivos grandes, la cuota gratuita de GitHub LFS es de solo 1 GB, por lo que tus 4 GB generarían un fallo de cuota).

User prompt: no se que pasa

Response: En la imagen `image_44f83c.png`, lo que está ocurriendo es que estás intentando ejecutar el comando `git commit` repetidamente, pero Git te está avisando que **el commit ya se creó con éxito.**

Si observas la terminal, el sistema te indica: `Your branch is ahead of 'origin/main' by 1 commit`. Esto significa que los archivos de tu Fase 3.1 ya fueron empaquetados y guardados correctamente en tu historial local. Al volver a lanzar el comando `git commit`, Git simplemente te responde `nothing added to commit` porque no hay ninguna modificación nueva desde tu último guardado.

Para avanzar, solo te falta ejecutar el comando de envío para sincronizar ese paquete con GitHub. Escribe esto en tu terminal:

```
git push origin main
```

(Nota: El archivo en texto rojo "Configuración de Proyectos Xiima-Labs.pdf" simplemente indica que está sin rastrear. Puedes ignorarlo con total tranquilidad por ahora, ya que no interferirá con tu subida).

User prompt: creo que tocara reiniciar .git porque hay errores mmm

Response: Tienes toda la razón y es la decisión más estratégica en este punto. Lo que está ocurriendo es que, en tus intentos anteriores, la carpeta oculta `.git` alcanzó un tamaño de 4.34 GB. Git "memorizó" los archivos multimedia de la carpeta `Meeting Recordings`, y aunque intentes subir el proyecto por partes ahora, Git sigue intentando enviar ese historial masivo, chocando contra el límite de 2.00 GiB de GitHub.

Para limpiar esa memoria sin perder tu progreso en la nube (Fases 1 y 2) ni borrar tus archivos físicos locales, vamos a reiniciar `.git`, sincronizarlo con lo que ya está en GitHub y bloquear los videos pesados para siempre.

Ejecuta este protocolo paso a paso en tu terminal Git Bash:

Paso 1: Formatear la memoria local y reconectar con GitHub

```
# 1. Eliminar el historial local sobrecargado (Tus archivos físicos están a salvo)
rm -rf .git

# 2. Inicializar Git desde cero
git init
git branch -M main

# 3. Reconectar con tu repositorio en GitHub
git remote add origin git@github.com:PoligranBitcoinClub/docs-central-pbc.git

# 4. Descargar el historial de lo que YA subiste (Fases 1 y 2)
git fetch origin

# 5. Sincronizar tu carpeta local con la nube (esto evita errores de historial)
git reset --mixed origin/main
```

Paso 2: Crear un "escudo" contra los videos pesados Para evitar tener que subir archivo por archivo o cometer errores en el futuro, vamos a decirle a Git que ignore la carpeta de videos por completo.

```
# 6. Añadir la ruta de los videos al archivo .gitignore
echo "docs/02-estado-del-arte/guia-cbc/Meeting Recordings/" >> .gitignore
```

Paso 3: Subir la Fase 3 completa (sin riesgo de bloqueo) Ahora que los videos están bloqueados y tu `.git` está limpio, puedes subir toda la carpeta `docs` de un solo golpe con total seguridad.

```
# 7. Indexar el archivo .gitignore actualizado y la carpeta docs
git add .gitignore
git add docs/

# 8. Crear el empaquetado final
git commit -m "docs: fase 3 completada - documentacion sin videos pesados"

# 9. Enviar a la nube de GitHub
git push -u origin main
```

¿Qué hacer con los videos de "Meeting Recordings"? Como charlamos previamente, la mejor práctica de ingeniería es no alojar esos 4 GB de grabaciones en GitHub. Súbelos a un servicio de alojamiento en la nube (Google Drive institucional, OneDrive o YouTube oculto) y luego simplemente crea un archivo de texto llamado `README.md` dentro de la carpeta `Meeting Recordings` con los enlaces a esos videos.

User prompt: Todos los códigos estuvieron bien menos el ultimo. NO SE QUE PASA

Response: El error `client_loop: send disconnect: Connection reset by peer/s` no es un fallo en tus archivos ni en los comandos anteriores; se debe a que la sesión SSH o la red sufrió una micro-interrupción mientras transfería el paquete comprimido de 120 objetos a los servidores de GitHub.

La buena noticia es que el reinicio de `.git` funcionó: pasaste de un historial bloqueado de más de 4 GB a un paquete limpio de solo 120 archivos. Para evitar que la conexión se cierre por inactividad durante el envío, ejecuta los siguientes pasos en tu terminal Git Bash:

Paso 1: Evitar que SSH corte la conexión (Keep-Alive) Ejecuta este bloque para forzar a SSH a enviar señales de mantenimiento (*heartbeat*) cada 5 segundos:

```
mkdir -p ~/.ssh
echo "Host github.com" >> ~/.ssh/config
echo "  ServerAliveInterval 5" >> ~/.ssh/config
echo "  ServerAliveCountMax 100" >> ~/.ssh/config
chmod 600 ~/.ssh/config
```

Paso 2: Ajustar el búfer de transmisión de Git Aumenta la memoria de envío para evitar el fallo de tubería rota (`Broken pipe`):

```
git config --global http.postBuffer 524288000
git config --global core.compression 1
```

Paso 3: Reintentar el envío Con el canal blindado contra desconexiones, lanza nuevamente el comando de sincronización:

```
git push -u origin main
```

Nota: Si la red Wi-Fi en la que estás conectado bloquea el puerto SSH de forma persistente, cambia el transporte a HTTPS ejecutando `git remote set-url origin [https://github.com/PoligranBitcoinClub/docs-central-pbc.git](https://github.com/PoligranBitcoinClub/docs-central-pbc.git)` y vuelve a lanzar el `git push`.

User prompt: Ya se creo el contenedor(carpeta "docs") en la nube de GitHub pero faltan algunos archivos y subcarpetas como "03-tesis" y "04-workshops". NO SE PORQUE "guia-pbc" la marca diferente y no como una subcarpeta dentro de "02-estado-del-arte" en la nube de GitHub, lo mismo con "05-registros-historicos" y la subcarpeta "chats_gemini" ademas estas subcarpetas estan en color blanco y no azul y en la parte derecha estan a la derecha y no debajo de la carpeta anterior sin identacion y eso me confunde. ADEMAS QUEDA FALTANDO SUBIR "Meeting Recordings" POR SI ESTOY EN OTRO COMPUTADOR Q NO SEA EL MIO(LOCAL). PERO YA HEMOS AVANZADO.

Response: 1. ¿Por qué no aparecen las carpetas `03-tesis` y `04-workshops` en GitHub?

Explicación: Git **no rastrea ni sube carpetas que estén completamente vacías.** Si en tu computador local esas carpetas no contienen ningún archivo adentro, Git no las incluye en el *commit* ni en el *push*.

Solución: Si quieres que las carpetas aparezcan en GitHub aunque aún no tengan contenido definitivo:

1. Abre las carpetas `03-tesis` y `04-workshops` en tu computador.
2. Crea dentro de cada una un archivo vacío llamado `.gitkeep` (o un archivo `README.md` corto).
3. Luego ejecuta en tu terminal Git Bash:

```
git add docs/03-tesis docs/04-workshops
git commit -m "docs: estructura de carpetas tesis y workshops"
git push origin main
```

2. ¿Por qué `02-estado-del-arte/guia-cbc` y `05-registros-historicos/chats_gemini` aparecen unidos en una sola línea?

Explicación: Esto no es un error de tu repositorio; es una función visual de la interfaz web de GitHub llamada **"Plegado de carpetas" (Folder Collapsing / Compact Directory View).**

- **¿Cómo funciona?** Cuando una carpeta (por ejemplo, `02-estado-del-arte`) contiene **únicamente una sola subcarpeta** (como `guia-cbc`) y ningún otro archivo al mismo nivel, GitHub las combina visualmente en una sola fila compacta (`02-estado-del-arte/guia-cbc`) para ahorrar espacio y clics al navegar.
- **¿Por qué se ven en blanco/azul o sin sangría?** En la barra lateral o en el panel de archivos de GitHub, las rutas combinadas se muestran en línea continua.

¿Cómo hacer que se vuelva a ver con la estructura de subcarpetas tradicional? Tan pronto como agregues un archivo suelto (por ejemplo, un `README.md` o una matriz en Excel) directamente dentro de `02-estado-del-arte` (al lado de `guia-cbc`), GitHub romperá la línea combinada y mostrará `02-estado-del-arte` como una carpeta normal desplegable con sangría.

3. Estrategia para subir y acceder a "Meeting Recordings" desde cualquier computador

Dado que la carpeta `Meeting Recordings` contiene archivos de video/audio que en conjunto superan las 4 GB, existen dos alternativas para gestionarla:

Opción A: La mejor práctica de ingeniería (Nube externa + Enlace en GitHub)

Para no saturar el repositorio y evitar bloqueos por tamaño de paquete en Git:

1. Sube la carpeta `Meeting Recordings` a **Google Drive**, **OneDrive** o a un canal no listado/privado de **YouTube.**
2. Dentro de la carpeta `guia-cbc` en GitHub, crea o edita un archivo `README.md` con los enlaces directos a esa carpeta en la nube.
3. De esta forma, cuando estés en cualquier otro computador, abres tu repositorio de GitHub, haces clic en el enlace y descargas o visualizas las grabaciones de inmediato.

Opción B: Subida directa a GitHub (Solo si los videos son livianos)

Si decides subirlos directamente a Git:

- Asegúrate de que **ningún archivo individual supere los 100 MB** (límite máximo estricto de GitHub).
- Quita la ruta de la carpeta del archivo `.gitignore` si la tenías bloqueada.
- Sube los videos de uno en uno para no saturar la conexión:

```
git add "docs/02-estado-del-arte/guia-cbc/Meeting Recordings/video_1.mp4"
git commit -m "docs: subiendo grabacion 1"
git push origin main
```
